APUNTES. MODULO 5. Control de errores

Aseguramiento de la calidad y pruebas de software con .NET

1. Objetivos del tema

• Distinguir estrategias básicas para resolver problemas y validar soluciones.

• Conocer los tipos de pruebas: unitarias, integración, sistema y navegación.

• Entender por qué el control de errores (try-catch-finally) es clave para la calidad.

• Conocer herramientas de automatización: NUnit, Mocks y Selenium.

2. Diferenciación y uso de estrategias de resolución de problemas

Resolver un problema no es solo "que funcione una vez". La calidad exige que funcione con distintos datos, que falle de forma controlada y que sea fácil de mantener.

Estrategias sencillas:

• **Divide y vencerás:** Rompe el problema grande en partes pequeñas (métodos/funciones).

• **Casos típicos y extremos:** Prueba con datos normales y también con vacíos, negativos o muy grandes.

• **Validación de entradas:** Comprueba null, rangos, formatos antes de ejecutar la lógica.

• **Mensajes y registro de errores:** Si falla, muestra un mensaje claro y/o registra la causa.

• **Repetibilidad:** Si falla, que se pueda repetir con los mismos pasos/datos.

Estas estrategias se convierten en pruebas: comprobar que cada parte funciona y que todo junto también.

3. Tipos de pruebas (qué son y cuándo se usan)

3.1 Pruebas unitarias

Comprueban una unidad pequeña (un método) de forma aislada. Son rápidas y se ejecutan muchas veces.

3.2 Pruebas de integración

Comprueban que varias piezas funcionan juntas (por ejemplo, formulario + base de datos).

3.3 Pruebas de sistema

Comprueban el sistema completo como lo usaría un usuario final (flujo general, configuración, pantallas).

3.4 Pruebas de navegación

Comprueban la navegación entre pantallas/ventanas: menús, botones, abrir/cerrar formularios, etc.

4. Control de errores: TRY–CATCH–FINALLY (parte principal del tema)

Una app de calidad no solo funciona cuando todo va bien: también maneja errores sin "romperse". En .NET se usa try–catch–finally para controlar errores y limpiar recursos.

4.1 ¿Qué hace cada bloque?

- **try:** Código que puede fallar (BD, archivos, conversiones...).
- **catch:** Qué hacemos si falla (mensaje, registrar, devolver valor...).
- **finally:** Se ejecuta SIEMPRE: cerrar conexión, liberar archivo, reactivar botones...

4.2 Ejemplo práctico grande (WinForms + SQL + validación)

Objetivo: cargar datos en un DataGridView desde SQL Server, con validación y manejo de errores.

Código (para copiar y pegar):

```
// Ejemplo MUY didáctico: cargar una tabla en un DataGridView
// Controles en el formulario:
// - TextBox txtServidor (ej: .\SQLEXPRESS)
// - TextBox txtBD       (ej: bdprueba01)
// - Button  btnCargar
// - DataGridView grid

using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

private void btnCargar_Click(object sender, EventArgs e)
{
    // Evita doble clic mientras carga
    btnCargar.Enabled = false;

    SqlConnection cn = null;

    try
    {
```

```
        // 1) Validaciones sencillas
        if (string.IsNullOrWhiteSpace(txtServidor.Text))
            throw new Exception("El servidor está vacío. Ejemplo: .\\SQLEXPRESS");

        if (string.IsNullOrWhiteSpace(txtBD.Text))
            throw new Exception("La base de datos está vacía. Ejemplo: bdprueba01");

        // 2) Cadena de conexión
        string cs =
            "Server=" + txtServidor.Text + ";" +
            "Database=" + txtBD.Text + ";" +
            "Trusted_Connection=True;" +
            "TrustServerCertificate=True;";

        // 3) Conexión
        cn = new SqlConnection(cs);

        // 4) Abrir (puede fallar)
        cn.Open();

        // 5) Consulta (puede fallar si la tabla no existe)
        string sql = "SELECT idprovincia, provincia FROM provincias ORDER BY
idprovincia";

        using (SqlCommand cmd = new SqlCommand(sql, cn))
        using (SqlDataAdapter da = new SqlDataAdapter(cmd))
        {
            DataTable dt = new DataTable();
            da.Fill(dt);
            grid.DataSource = dt;
        }

        MessageBox.Show("Datos cargados correctamente ");
    }
    catch (SqlException exSql)
    {
        // Errores de SQL Server
        MessageBox.Show("Error de SQL Server:\n" + exSql.Message);
    }
    catch (Exception ex)
    {
        // Otros errores (validación, etc.)
        MessageBox.Show("Error:\n" + ex.Message);
    }
    finally
    {
        // SIEMPRE se ejecuta: limpiar recursos
        if (cn != null && cn.State == ConnectionState.Open)
            cn.Close();

        // Rehabilitar botón pase lo que pase
        btnCargar.Enabled = true;
    }
}
```

4.3 Cómo explicarlo en clase (paso a paso)

1. Validamos entradas antes de conectar (calidad).

2. try: intentamos abrir conexión y hacer el SELECT.

3. catch(SqlException): tratamos errores de base de datos.

4. catch(Exception): tratamos el resto (por ejemplo, validación).

5. finally: cerramos conexión y reactivamos el botón SIEMPRE.

Idea clave para memorizar: try = intento | catch = si falla | finally = siempre.

5. Práctica rápida

1) Escribe mal el nombre de la BD y observa qué catch se ejecuta.

2) Cambia la consulta a una tabla inexistente y observa el error.

3) Explica por qué el finally es imprescindible (cierre de conexión y reactivar botón).

6. Automatización de pruebas

Herramientas a valorar:

No las vemos en este curso, pero se enuncian por si el alumno quiere ampliar el tema más adelante

• NUnit (pruebas unitarias)

• Mocks (objetos simulados)

• Selenium (automatización en navegador)

GLOSARIO DE CULTURA GENERAL DE ERRORES

¿Qué es un error informático?

Un error informático es un error o mal funcionamiento que se produce en un sistema informático y que provoca un comportamiento inesperado o incorrecto. Puede manifestarse de diversas formas, desde fallos de software hasta fallos de hardware, y puede provocar el bloqueo de programas, la pérdida de datos o incluso el fallo del sistema.

¿Por qué se producen los errores informáticos?

Los errores informáticos pueden deberse a varias razones. A veces, los errores se producen por fallos o defectos en el código del software. Otras veces, los errores pueden deberse a problemas de hardware, como componentes defectuosos o conexiones inadecuadas. Además, los errores pueden deberse a fallos del usuario, como la introducción de datos incorrectos o una configuración inadecuada del sistema.

¿Cómo puedo identificar un error informático?

La identificación de un error informático suele hacerse observando los síntomas. Estos pueden incluir programas que se congelan o se bloquean, mensajes de error que aparecen en la pantalla, un rendimiento lento del sistema, reinicios inesperados o un comportamiento inusual de los dispositivos de hardware. Prestando atención a estos signos, puede hacerse una idea de si se ha producido un error informático.

¿Qué es un error de pantalla azul?

Un error de pantalla azul, también conocido como pantalla azul de la muerte (BSOD), es un error crítico que se produce en los sistemas operativos Windows. Cuando se produce un error de pantalla azul, la pantalla del ordenador se vuelve azul y muestra un mensaje de error. Estos errores suelen estar causados por problemas graves del sistema o del hardware, como conflictos de controladores, memoria defectuosa o corrupción de archivos del sistema. Para resolver un error de pantalla azul, es posible que tenga que actualizar los controladores, ejecutar diagnósticos de hardware o reparar el sistema operativo.

¿Qué es un error de kernel panic?

Un error de kernel panic, también conocido como "kernel panic" o "error de pánico", es un tipo de error que ocurre en sistemas operativos basados en Unix, como Linux. Significa un error crítico del sistema del que el sistema operativo no puede recuperarse con seguridad. Cuando se produce un error de pánico del kernel, el sistema muestra una pantalla con información de diagnóstico y normalmente requiere un reinicio manual. Los errores de kernel panic pueden deberse a problemas de hardware o software, como controladores defectuosos, hardware incompatible o archivos de sistema dañados.

¿Los errores informáticos pueden provocar la pérdida de datos?

Sí, los errores informáticos a veces pueden provocar la pérdida de datos. Por ejemplo, si un programa se bloquea mientras estás trabajando en un documento y no lo has guardado, puedes perder todos los cambios que no hayas guardado. Del mismo modo, los errores en los dispositivos de almacenamiento o en los sistemas de archivos pueden corromper o borrar archivos. Para minimizar el riesgo de pérdida de datos, es importante hacer copias de seguridad periódicas de los archivos y datos importantes.

¿Se pueden solucionar los errores informáticos?

Sí, los errores informáticos suelen tener solución. Muchos errores pueden solucionarse mediante pasos de solución de problemas, actualizaciones de software o aplicando parches proporcionados por los desarrolladores de software. En algunos casos, es posible que tenga

que volver a instalar un programa o actualizar los controladores de dispositivos para solucionar los errores. Sin embargo, puede haber casos en los que problemas más complejos requieran asistencia profesional o reparaciones de hardware para resolver completamente el error.

¿Cómo puedo prevenir los errores informáticos?

Aunque no sea posible evitar todos los errores informáticos, hay varias medidas que puede tomar para minimizar su aparición. Mantener el sistema y el software al día con los últimos parches y actualizaciones puede ayudar a solucionar problemas conocidos y vulnerabilidades de seguridad. Además, utilizar un software antivirus fiable y realizar análisis periódicos del sistema puede ayudar a detectar y eliminar cualquier malware o virus que pudiera causar errores. Ser precavido al instalar software nuevo, evitar sitios web sospechosos y practicar hábitos informáticos seguros también puede contribuir a prevenir errores.

¿Cuáles son los tipos más comunes de errores informáticos?

Existen varios tipos comunes de errores informáticos con los que puede encontrarse:

- **Errores de sintaxis:** Se producen cuando hay un error en la sintaxis del código, lo que impide que el ordenador lo ejecute correctamente.
- **Errores de lógica:** Los errores lógicos ocurren cuando hay un fallo en el flujo lógico de un programa, haciendo que produzca resultados incorrectos o inesperados.
- **Errores de ejecución:** Estos errores se producen durante la ejecución de un programa y pueden hacer que se bloquee o se comporte de forma anormal. Suelen deberse a problemas como la división por cero o el acceso a posiciones de memoria no válidas.
- **Errores de hardware:** Los errores de hardware implican un mal funcionamiento o fallos en los componentes de hardware del ordenador, como el disco duro (HD), la memoria de acceso aleatorio (RAM) o la tarjeta gráfica. Estos errores pueden provocar caídas del sistema, corrupción de datos o mal funcionamiento del dispositivo.
- **Errores de comunicación:** Los errores de comunicación se producen cuando hay problemas con la transmisión de datos entre diferentes componentes o dispositivos. Esto puede provocar la pérdida de datos, fallos de conexión o velocidades de red lentas.

¿Cómo puedo corregir un error sintáctico en mi código?

Corregir un error de sintaxis en el código suele implicar identificar la línea o sección del código en la que se produce el error y corregir el error de sintaxis. Los errores de sintaxis más comunes son la falta de punto y coma, paréntesis o corchetes mal colocados y palabras clave mal escritas. Revisar el mensaje de error proporcionado por el entorno de programación a menudo permite localizar el error, lo que facilita su corrección.

¿Cuál es la diferencia entre un fallo de software y un error informático?

Aunque los términos "fallo de software" y "error informático" suelen utilizarse indistintamente, tienen significados ligeramente diferentes. Un fallo de software se refiere a un defecto o error en el código del programa, que puede hacer que funcione incorrectamente o produzca resultados inesperados. Por otro lado, un error informático es un término más amplio que engloba cualquier error o mal funcionamiento dentro de un sistema informático, que puede incluir tanto fallos de software como problemas de hardware o errores del usuario.

¿Qué papel desempeña el tratamiento de errores en la programación?

El tratamiento de errores es una parte esencial de la programación que consiste en anticipar y gestionar los errores. Permite a los programadores gestionar posibles errores o excepciones que puedan producirse durante la ejecución del programa. Mediante la implementación de mecanismos de gestión de errores, como los bloques try-catch, los programadores pueden detectar y responder a los errores de forma controlada, evitando fallos y la finalización inesperada del programa.

¿Los errores informáticos pueden ser beneficiosos o creativos?

Aunque los errores informáticos suelen verse como sucesos no deseados, hay casos en los que pueden ser beneficiosos o incluso dar lugar a resultados creativos. Por ejemplo, algunas obras de arte o música generadas por ordenador se crean introduciendo intencionadamente errores o fallos en el proceso. Además, a veces los errores pueden revelar problemas subyacentes o vulnerabilidades en el software, lo que conduce a mejoras y sistemas más seguros.